

Solving the momentum and pressure equations

F. Auteri

Dipartimento di Scienze e Tecnologie Aerospaziali
Politecnico di Milano

Course of High Performance Scientific Computing in Aerospace,
Module A, A. Y. 2025/26.

Outline

- 1 Introduction
- 2 Building the linear systems
- 3 The linear system: serial solution
- 4 Parallel solution: the Schur complement
- 5 References

In this lecture

In this lecture we will:

- explore a bit more the discretisation of the momentum equation;
-

Outline

- 1 Introduction
- 2 Building the linear systems**
- 3 The linear system: serial solution
- 4 Parallel solution: the Schur complement
- 5 References

The direction splitting for the momentum equation

Defining the right hand side of the system as:

$$\hat{\mathbf{g}}^{n+1/2} = \mathbf{f}^{n+1/2} - \nabla p_{\star}^{n+1/2} - \frac{\nu}{2k} \mathbf{u}^n + \frac{\nu}{2} (\partial_{xx} \eta^n + \partial_{yy} \zeta^n + \partial_{zz} \mathbf{u}^n).$$

the modified direction-splitting time step for the momentum equation reads:

$$\xi^{n+1} - \mathbf{u}^n = \frac{\Delta t}{\beta} \hat{\mathbf{g}}^{n+1/2}$$

$$\eta^{n+1} - \xi^{n+1} - \gamma \partial_{xx} (\eta^{n+1} - \eta^n) = 0$$

$$\zeta^{n+1} - \eta^{n+1} - \gamma \partial_{yy} (\zeta^{n+1} - \zeta^n) = 0$$

$$\mathbf{u}^{n+1} - \zeta^{n+1} - \gamma \partial_{zz} (\mathbf{u}^{n+1} - \mathbf{u}^n) = 0$$

which allows also the velocity to be advanced in $O(N^3)$ operations.

A more convenient form of the momentum equation

This step can be rewritten in the following, more convenient form

$$\boldsymbol{\xi}^{n+1} = \boldsymbol{u}^n + \frac{\Delta t}{\beta} \hat{\boldsymbol{g}}^{n+1/2}$$

$$(I - \gamma \partial_{xx})(\boldsymbol{\eta}^{n+1} - \boldsymbol{\eta}^n) = \boldsymbol{\xi}^{n+1} - \boldsymbol{\eta}^n$$

$$(I - \gamma \partial_{yy})(\boldsymbol{\zeta}^{n+1} - \boldsymbol{\zeta}^n) = \boldsymbol{\eta}^{n+1} - \boldsymbol{\zeta}^n$$

$$(I - \gamma \partial_{zz})(\boldsymbol{u}^{n+1} - \boldsymbol{u}^n) = \boldsymbol{\zeta}^{n+1} - \boldsymbol{u}^n$$

Other forms can be easily formulated, after the solver has been implemented and validated to optimise it.

The direction splitting for the pressure equation

The pressure step can be written in the following form:

$$\begin{aligned}(I - \partial_{xx})\psi &= -\frac{1}{\Delta t} \nabla \cdot \mathbf{u}^{n+1}, & \partial_x \psi|_{x=0,1} &= 0; \\ (I - \partial_{yy})\varphi &= \psi, & \partial_y \varphi|_{y=0,1} &= 0; \\ (I - \partial_{zz})\phi^{n+1/2} &= \varphi, & \partial_z \phi|_{z=0,1} &= 0.\end{aligned}$$

that is very similar to the velocity step. The matrices for the linear systems coincide if $\gamma = 1$.

Discretisation of the equations: x momentum

Let us discretise the left-hand side. Please remind that the **grids are staggered**. x component of the momentum, point $(i + 1/2, j, k)$.

x direction:

$$-\gamma \Delta x^{-2} \Delta \eta_{i-1/2,j,k}^x + (1 + 2\gamma \Delta x^{-2}) \Delta \eta_{i+1/2,j,k}^x - \gamma \Delta x^{-2} \Delta \eta_{i+3/2,j,k}^x$$

y direction:

$$-\gamma \Delta y^{-2} \Delta \zeta_{i+1/2,j-1,k}^x + (1 + 2\gamma \Delta y^{-2}) \Delta \zeta_{i+1/2,j,k}^x - \gamma \Delta y^{-2} \Delta \zeta_{i+1/2,j+1,k}^x$$

z direction:

$$-\gamma \Delta z^{-2} \Delta \mathbf{u}_{i+1/2,j,k-1}^x + (1 + 2\gamma \Delta z^{-2}) \Delta \mathbf{u}_{i+1/2,j,k}^x - \gamma \Delta z^{-2} \Delta \mathbf{u}_{i+1/2,j,k+1}^x$$

Discretisation of the equations: y momentum

Let us discretise the left-hand side. Please remind that the **grids are staggered**. x component of the momentum, point $(i, j + 1/2, k)$.

x direction:

$$-\gamma \Delta x^{-2} \Delta \eta_{i-1, j+1/2, k}^y + (1 + 2\gamma \Delta x^{-2}) \Delta \eta_{i, j+1/2, k}^y - \gamma \Delta x^{-2} \Delta \eta_{i+1, j+1/2, k}^y$$

y direction:

$$-\gamma \Delta y^{-2} \Delta \zeta_{i, j-1/2, k}^y + (1 + 2\gamma \Delta y^{-2}) \Delta \zeta_{i, j+1/2, k}^y - \gamma \Delta y^{-2} \Delta \zeta_{i, j+3/2, k}^y$$

z direction:

$$-\gamma \Delta z^{-2} \Delta \mathbf{u}_{i, j+1/2, k-1}^y + (1 + 2\gamma \Delta z^{-2}) \Delta \mathbf{u}_{i, j+1/2, k}^y - \gamma \Delta z^{-2} \Delta \mathbf{u}_{i, j+1/2, k+1}^y$$

Discretisation of the equations: z momentum

Let us discretise the left-hand side. Please remind that the **grids are staggered**. x component of the momentum, point $(i, j, k + 1/2)$.

x direction:

$$-\gamma \Delta x^{-2} \Delta \eta_{i-1,j,k+1/2}^z + (1 + 2\gamma \Delta x^{-2}) \Delta \eta_{i,j,k+1/2}^z - \gamma \Delta x^{-2} \Delta \eta_{i+1,j,k+1/2}^z$$

y direction:

$$-\gamma \Delta y^{-2} \Delta \zeta_{i,j-1,k+1/2}^z + (1 + 2\gamma \Delta y^{-2}) \Delta \zeta_{i,j,k+1/2}^z - \gamma \Delta y^{-2} \Delta \zeta_{i,j+1,k+1/2}^z$$

z direction:

$$-\gamma \Delta z^{-2} \Delta u_{i,j,k-1/2}^z + (1 + 2\gamma \Delta z^{-2}) \Delta u_{i,j,k+1/2}^z - \gamma \Delta z^{-2} \Delta u_{i,j,k+3/2}^z$$

Discretisation of the pressure equation

Let us discretise the left-hand side. Please remind that the pressure grid is the central one:

x direction:

$$-\Delta x^{-2} \Delta \psi_{i-1,j,k} + (1 + 2\Delta x^{-2}) \Delta \psi_{i,j,k} - \Delta x^{-2} \Delta \psi_{i+1,j,k}$$

y direction:

$$-\Delta y^{-2} \Delta \varphi_{i,j-1,k} + (1 + 2\Delta y^{-2}) \Delta \varphi_{i,j,k} - \Delta y^{-2} \Delta \varphi_{i,j+1,k}$$

z direction:

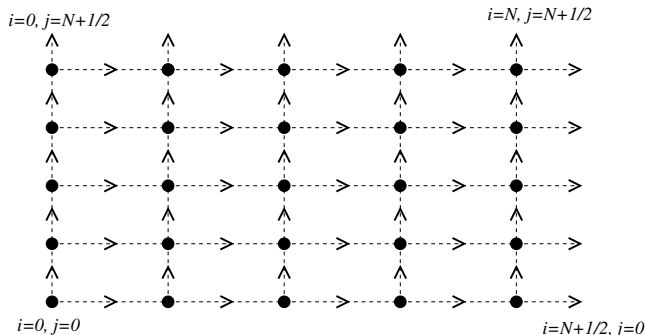
$$-\Delta z^{-2} \Delta \phi_{i,j,k-1} + (1 + 2\Delta z^{-2}) \Delta \phi_{i,j,k} - \Delta z^{-2} \Delta \phi_{i,j,k+1}$$

The matrix is the same as that for the momentum equation, if we set $\gamma = 1$, it can be produced by the same function.

Boundary conditions

The compute the matrices we need to:

- select the boundary conditions: **Dirichlet** conditions for **velocity** and homogeneous **Neumann** conditions for the **pressure**.
- choose the points where to impose the boundary conditions:



Dirichlet conditions on the left boundary for velocity

- Tangent components: set the matrix row to 0, but for 1 on the main diagonal, impose the b.c. value in the rhs.
- Normal component: the b.c. is imposed in the $i = 1/2$ (or $j = 1/2$, $k = 1/2$) using the incompressibility constraint.
 - We compute the two contributions coming from the components tangent to the boundary using the boundary values. For instance on the $j = 0$, $k = 0$ boundary we have

$$\begin{aligned}\frac{\partial u}{\partial x}|_{0,j,k} &= -\frac{\partial y}{\partial y}|_{0,j,k} - \frac{\partial w}{\partial z}|_{0,j,k} \\ &\approx -\frac{v|_{0,j+1/2,k} - v|_{0,j-1/2,k}}{\Delta y} - \frac{w|_{0,j,k+1/2} - w|_{0,j,k-1/2}}{\Delta z}\end{aligned}$$

- Then, at second order,

$$u_{1/2,j,k} = u_{0,j,k} + \frac{\partial u}{\partial x}|_{0,j,k} \Delta x / 2$$

Dirichlet conditions on the right boundary for velocity

- Normal component: as on the left side for the tangent components;
- Tangent components: use a ghost node ($N + 1$): use the b.c. to eliminate it from the equation for the point N , i.e. eliminate $u_{N+1,j,k}$ between:

$$au_{N-1,j,k} + bu_{N,j,k} + cu_{N+1,j,k} = f$$

and

$$u_{N-1,j,k}/2 + u_{N+1,j,k}/2 = u_{ex,j,k}$$

so that the equation for the point N becomes:

$$au_{N-1,j,k} + (b - c)u_{N,j,k} = f - 2cu_{ex,j,k}$$

Neumann conditions on the left boundary for pressure

- Introduce a ghost node (-1) and use the b.c. to eliminate it from the equation for the point 0: eliminate $p_{-1,j,k}$ between:

$$ap_{-1,j,k} + bp_{0,j,k} + cp_{1,j,k} = f$$

and

$$\frac{p_{1,j,k} - p_{-1,j,k}}{2\Delta x} = \frac{\partial p}{\partial x_{ex,j,k}}$$

so that the equation for the point N becomes:

$$bp_{0,j,k} + (c + a)p_{1,j,k} = f + 2\Delta xa \frac{\partial p}{\partial x_{ex,j,k}}$$

Neumann conditions on the right boundary for pressure

This is similar to the previous, but the stencil is shorter...

- Introduce a ghost node ($N + 1$) and use the b.c. to eliminate it from the equation for the point N : eliminate $p_{N+1,j,k}$ between:

$$ap_{N-1,j,k} + bp_{N,j,k} + cp_{N+1,j,k} = f$$

and

$$\frac{p_{N+1,j,k} - p_{N,j,k}}{\Delta x} = \frac{\partial p}{\partial x}_{ex,j,k}$$

so that the equation for the point N becomes:

$$ap_{N-1,j,k} + (b + c)p_{N,j,k} = f - \Delta x c \frac{\partial p}{\partial x}_{ex,j,k}$$

Outline

- 1 Introduction
- 2 Building the linear systems
- 3 The linear system: serial solution**
- 4 Parallel solution: the Schur complement
- 5 References

The linear systems

- By virtue of the adopted fractional-step method and of the second order finite difference scheme, we have to solve just tridiagonal linear systems.
- We can use Gaussian elimination to directly solve the systems (Thomas algorithm) or to factor the matrix and then back-substitute.
- For tridiagonal systems, Thomas algorithm reduces complexity to $O(n)$.
- Thomas' algorithm amounts to a forward and a backward sweep, both require $O(N)$ operations.

Tridiagonal system

We'll consider the tridiagonal system:

$$a_{i-1}x_{i-1} + b_i x_i + c_i x_{i+1} = f_i, \quad i = 1, \dots, n$$

For example, with $n = 4$:

$$\begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Thomas' algorithm

- Thomas' algorithm is made of a forward sweep (Gaussian elimination), it amounts to repeat two steps on each line, recursively:
 - Rescale the current equation to have one on the diagonal;
 - Eliminate the lower diagonal of the next equation (this operation changes the diagonal element and the right-hand side)
- The backward step is the back substitution, quite trivial since two diagonals are left.

Forward sweep

- 1 $c_0 = c_0/b_0; f_0 = f_0/b_0;$
- 2 For $i = 1, N$:
 - $b_i = b_i - a_{i-1}c_{i-1}; f_i = f_i - a_{i-1}f_{i-1}$
 - $c_i = c_i/b_i; f_i = f_i/b_i;$

The algorithm can then be simplified:

- 1 $c_0 = c_0/b_0; f_0 = f_0/b_0;$
- 2 For $i = 1, N$:
 - $c_i = c_i/(b_i - a_{i-1}c_{i-1}); f_i = (f_i - a_{i-1}f_{i-1})/(b_i - a_{i-1}c_{i-1});$

Backward sweep

- 1 $x_N = f_N;$
- 2 For $i = N - 1, 0:$
 - $x_i = f_i - c_i x_{i+1};$

Forward Elimination: example

We rewrite coefficients using recurrence formulas.

$$c_0 = \frac{-1}{2} = -0.5, \quad d'_1 = \frac{1}{2} = 0.5$$

$$c_1 = \frac{-1}{2 - (-1)(-0.5)} = -\frac{2}{3}, \quad d'_2 = \frac{0 - (-1)(0.5)}{1.5} = \frac{1}{3}$$

$$c_2 = \frac{-1}{2 - (-1)(-2/3)} = -\frac{3}{4}, \quad d'_3 = \frac{0 - (-1)(1/3)}{4/3} = \frac{1}{4}$$

$$c_3 \text{ not needed}, \quad d'_4 = \frac{1 - (-1)(1/4)}{2 - (-1)(-3/4)} = \frac{5}{5/4} = 1$$

Backward Substitution: example

Now solve for x_i backwards: **Example:**

$$x_3 = f_3 = 1$$

$$x_2 = f_2 - c_2 x_3 = \frac{1}{4} - \left(-\frac{3}{4}\right)(1) = 1$$

$$x_1 = f_1 - c_1 x_2 = \frac{1}{3} - \left(-\frac{2}{3}\right)(1) = 1$$

$$x_0 = f_0 - c_0 x_1 = \frac{1}{2} - (-0.5)(1) = 1$$

$$\Rightarrow \mathbf{x} = (1, 1, 1, 1)^T$$

Outline

- 1 Introduction
- 2 Building the linear systems
- 3 The linear system: serial solution
- 4 Parallel solution: the Schur complement**
- 5 References

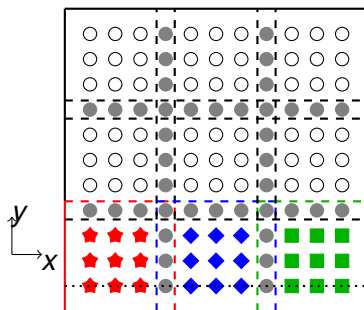
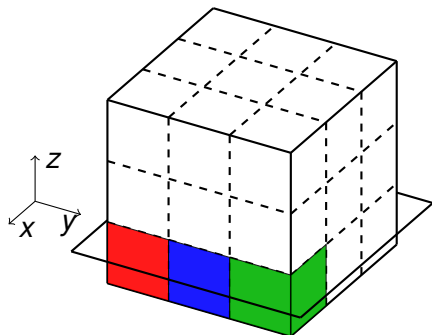
Parallelisation

- The Thomas algorithm is very efficient but...
- It is recursive, and then it is difficult to exploit the vectorial capabilities of the processor;
- For the same reason, it cannot be easily parallelised.
- Fortunately, the band is very narrow, and we can exploit this fact in the parallelisation, to make it efficient.
- We will use the technique of the Schur complement to parallelise the solution of the linear systems.

Partitioning the domain

The first step for the parallelisation is to partition the domain along the three directions in equal-size blocks (to balance loading).

The interface points are stored on both neighbouring processors.



Partitioning the domain: interface points

- The grid points in each block are divided into internal points x_i and shared interface points x_s ;
- For internal points, the right-hand side of the linear systems can be assembled locally;
- For the shared interface points, assembling the right-hand side requires some sort of communication;
- as such points are placed on the interfaces between the blocks and their stencil is shared between consecutive processors.
- Two consecutive blocks share interface points. Both solve the linear systems for such points in the directions parallel to the face: this small overhead highly reduces the communication.

Block form of the linear systems

- Tridiagonal linear systems can be easily solved using the Schur complement method;
- For clarity, we consider a single line in the x direction, split across N_{px} processors
- For each processor the unknowns u are divided in internal unknowns u_i and shared interface unknowns u_s .

Grouping together internal and interface unknowns, the system can be written in the block form:

$$\begin{bmatrix} A_{ii} & A_{is} \\ A_{si} & A_{ss} \end{bmatrix} \begin{bmatrix} u_i \\ u_s \end{bmatrix} = \begin{bmatrix} f_i \\ f_s \end{bmatrix}.$$

Structure of the blocks

- The reordered linear system is no longer tridiagonal;
- But the four blocks are very sparse

$$A_{ii} = \begin{bmatrix} \star & \star & & & & & \\ \star & \star & \star & & & & \\ & \star & \star & & & & \\ & & & \diamond & \diamond & & \\ & & & \diamond & \diamond & \diamond & \\ & & & & \diamond & \diamond & \\ & & & & & \blacksquare & \blacksquare \\ & & & & & \blacksquare & \blacksquare & \blacksquare \\ & & & & & & \blacksquare & \blacksquare \end{bmatrix}$$

Red stars, blue diamonds and green squares refer to the internal grid points, grey circles indicate the shared interface grid points.

Structure of the blocks

- The reordered linear system is no longer tridiagonal;
- But the four blocks are very sparse

$$A_{is} = \begin{bmatrix} & & & & \\ & \star & & & \\ & \diamond & & & \\ & & & & \\ & & & \diamond & \\ & & & & \blacksquare \end{bmatrix} \quad A_{si}^T = \begin{bmatrix} & & & & \\ & \circ & & & \\ & \circ & & & \\ & & & & \\ & & & \circ & \\ & & & \circ & \end{bmatrix} \quad A_{ss} = \begin{bmatrix} \circ & & \\ & & \\ & & \circ \end{bmatrix}.$$

Red stars, blue diamonds and green squares refer to the internal grid points, grey circles indicate the shared interface grid points.

Use block Gaussian elimination on the reordered matrix

Please notice:

- A_{ij} is tridiagonal
 - A_{ss} is diagonal
 - A_{is} and A_{si} have just two nonzero elements for each block.
- ⇒ Use block Gaussian elimination to reduce the linear system to triangular form.
- The solution reduces to the successive solution of two linear systems: a small one:

$$S u_s = (A_{ss} - A_{si} A_{ij}^{-1} A_{is}) u_s = f_s - A_{si} A_{ij}^{-1} f_i,$$

→ and a big one:

$$A_{ij} u_i = f_i - A_{is} u_s,$$

S is the Schur complement and it is itself tridiagonal!

The algorithm

The algorithm reads:

- 1 Preprocessing
 - Assemble the tridiagonal matrix $S := (A_{ss} - A_{si}A_{ij}^{-1}A_{is})$
- 2 Runtime
 - Compute right-hand side for internal points f_i
 - Compute processor's contribution to the right-hand side of the equation for the shared interface unknowns: $f_s - A_{si}A_{ij}^{-1}f_i$
 - Assemble via communication all processors' contributions to $f_s - A_{si}A_{ij}^{-1}f_i$
 - Solve for shared interface unknowns u_s
 - Compute $f_i - A_{is}u_s$
 - Solve for internal unknowns u_i

Outline

- 1 Introduction
- 2 Building the linear systems
- 3 The linear system: serial solution
- 4 Parallel solution: the Schur complement
- 5 References**

Some references

- A. Chiarini, M. Quadrio, F. Auteri, A direction-splitting Navier-Stokes solver on co-located grids, Journal of Computational Physics, Volume 429, 2021, 110023.